

Lab4.5 Report

Baseline

Size	S	Time (ms)	GFLOPS	L2 relative error
4096	2	71.6008	1919.52	2.192e-5
4096	4	232.8551	590.23	3.397e-10
4096	6	494.5247	277.92	5.685e-15
4096	8	854.9332	160.76	2.150e-15
8192	2	451.1711	2437.02	2.192e-5
8192	4	1602.4298	686.15	3.398e-10
8192	6	3496.8838	314.43	6.075e-15
8192	8	6107.3491	180.03	3.026e-15

进一步 profile 结果表明，baseline 在计算过程中反复进行 D2H, H2D 通讯以及同步，这些都是毫无必要的。

此外，nsys 表明 NN INT8 GEMM kernel 发射了 64 instances，占约 63% 时间（1001ms）。进一步分析发现，它单个 kernel 就耗时 16ms，reg 165，occupancy 17%，而且最重要的是根本就没有调用 Tensor Core。这是最大瓶颈，说明我们应该重点优化。

并且，调用 73 次 cudaFree，进一步阅读源码，发现它每次都在显存中申请空间后释放，累计消耗 142ms。

那么初步 profile 结果表明，我们应该先优化内存访问，然后考虑重点由于 INT8 GEMM，主要是要让 Tensor Core 忙起来。

访存优化

GPU 算缩放系数

baseline 的缩放系数是把矩阵从显存拷回内存，由 CPU 去算的。纯粹多此一举。

写一个新 kernel 叫 maxabs_pair_kernel，直接算最大值，然后用另一个 prepare_scales_kernel 直接算出缩放系数，这样 D2H 就消失了。

批处理 split

原本对于 S 个 split 分成 S 个 kernel 来生成，这会导致矩阵被重复读取 S 遍，非常不好。

所以我们直接合并到一起（这算批处理还是 fusion）。profile 结果可以确认量化 kernel launch 从 25->2。

减少 Malloc&Free

之前说了 profile 表明我们每算一对 INT8 GEMM 就申请一块临时空间，不仅会让 CPU 忙起来，而且还会触发 GPU CPU 同步，浪费时间。

我们瓶颈又不在显存，所以我们直接申请一块 2GiB 大小的 grow-only workspace，后面所有中间产物都直接往这里放。

profile 确认不再进行显示 cudaMalloc/cudaFree

Tensor Core

原来完全不用 Tensor Core 太愚蠢了，相当于我们费劲把 FP64 变成 INT8 想利用计算单元，但是根本没用到。

baseline 中调用的是 cublasGemmEx，我们改用 cutlass 的 Tensor Core 调度。并且在量化 kernel 写 INT8 数据时直接让它写成 Tensor Core 需要的 TN with padding 布局。

这样我们单次 GEMM 从 11.22ms->1.42ms，提升了 7.88x，说明命中了 Tensor Core。

算法优化

按对角线合并

这是数学角度观察到的。

注意到，我们计算 $\sum_{i=0}^{S-1} \sum_{j=0}^{S-1} (s_i^A \times s_j^B) A_i B_j$ ，并且前面的系数其实只取决于 $i + j$ 的值。

那我们与其每次进行完 INT8 GEMM 之后都立刻乘缩放系数还原回 FP64 加回去，不如我们把 $i + j$ 相同的暂存下来，最后一块还原会 FP64 一块加回去。

目前已经能跑到 2650GFLOPS 左右 (S=8)，还差一点。

裁剪

因为权重按照 254^{-d} 衰减，所以可以预料的是对稍微大一些的 d 我们就没必要算了。

实测按不同 splits 取不同 D。

S=2 取 D=3 (全保留)，S=4 取 D=5，S=6 取 D=6，S=8 取 D=6。

取得了 1.25x-2.66x 的加速，并且精度误差仍然在同一数量级。

GEMM 数学合并

注意到，对于一个固定的 d ，比如 $d = 3$ ，我们要算的是 $A_0 B_3 + A_1 B_2 + A_2 B_1 + A_3 B_0$ ，会调用四次 GEMM。

然而从数学上讲，我们可以做如下的拼接：

$$A = \begin{pmatrix} A_0 & A_1 & A_2 & A_3 \end{pmatrix}$$
$$B = \begin{pmatrix} B_3 \\ B_2 \\ B_1 \\ B_0 \end{pmatrix}$$

这样只要计算 AB 就可以在一次 GEMM 中得到结果。当然我们并没有实际减少计算量，只是进行了 fusion 罢了。

大约提升了 6% 左右。目前 S6S8 已经压过 3000 线 (3143GFLOPS/3000.93GFLOPS) , 但是没什么余量, 还得继续优化。

这里其实还有可以扣的地方。

因为我们把矩阵沿 k 维度拼起来, 得到了特别长、特别宽的两个矩阵乘, cuBLAS 默认算法并不适合这种形状。

所以通过 CUBLAS_GEMM_AUTOTUNE 宏, 让 cuBLAS 自己选择哪个 kernel 最合适。

Size/S	concat + algo 5	concat + AUTOTUNE	提升
4096/S6	3143.27	3368.76	+7.2%
4096/S8	3000.93	3251.48	+8.4%
8192/S6	3721.32	3949.16	+6.1%
8192/S8	3620.47	3853.22	+6.4%

又取得了 6%-8% 的优化。目前均过线, 可以结束了。